SOKOINE UNIVERSITY OF AGRICULTURE

School of Engineering and Technology
Department of Agricultural Engineering

AGE 219: Basics of Computer Programming

Lecture Compendium 7: File Handling and Data Import — Bridging Code and Persistent Reality

---

Prepared by:

Dr. Kadeghe Fue, PhD, P.Eng (T)

Target Audience: Engineering Students (Agricultural, Irrigation, Post-Harvest, and related fields)
Primary Tools: Python 3.x and Google Colaboratory
Location: Morogoro, Tanzania

Programming without file handling is like a tractor without a trailer; you can generate power, but you cannot move or store the harvest. This lecture introduces persistence—the ability to save your program's outputs and import data from external sources such as weather stations, machinery data-loggers, and GIS databases.

Contents

1 Introduction: The Concept of Persistence **2**
1.1 Standard File Formats in Agricultural Engineering 2

2 The Mechanics of File I/O in Python **2**
2.1 Opening Files: The `open()` Function 2
2.1.1 File Modes 2
2.2 The "With" Statement: Professional Best Practice 3

3 Reading and Writing Text Files **3**
3.1 Writing Data 3
3.2 Reading Data 3

4 Working with CSV Files: The Agricultural Standard **3**
4.1 The `csv` Module 3
4.2 Reading a CSV into Data Structures 4

5 Google Colab: Getting Data into the Cloud **4**
5.1 Method 1: Manual Upload (Temporary) 4
5.2 Method 2: Mounting Google Drive (Permanent) 4
5.3 Method 3: Direct Upload via Code 4

6 Case Study 1: Parsing Weather Station Logs to Identify Frost Events **5**

7 Case Study 2: Machinery Telematics Report Generator **5**

8 Robust Programming: Error Handling in File I/O **6**

9 Best Practices for Engineering Data Import **6**

10 Summary **6**

11 Glossary of Terms **7**

12 Extended Self-Assessment Exercises **8**
12.1 Exercise 1: Simple Logger 8
12.2 Exercise 2: Field Coordinate Export 8
12.3 Exercise 3: The Corrupted Log Recovery 8
12.4 Exercise 4: Colab Path Logic 8

1 Introduction: The Concept of Persistence

Up to this point in AGE 219, every program we have written has been ”volatile.” This means that as soon as the Python script finishes running or the Google Colab runtime is disconnected, all the data stored in our variables, lists, and dictionaries is wiped from the computer’s RAM (Random Access Memory).

In professional engineering, this is unacceptable. An irrigation engineer needs to save daily water logs for annual reporting. A machinery researcher needs to import thousands of engine data points collected over weeks of field trials.

File Handling is the mechanism that allows us to:

1. **Write (Save):** Export data from RAM to a permanent storage device (Hard Drive, SSD, or Cloud Storage).
2. **Read (Load):** Import data from a storage device into the program’s memory for analysis.

1.1 Standard File Formats in Agricultural Engineering

As an engineer, you will encounter various file types. Python can handle almost all of them:

- **.txt (Plain Text):** Simple logs, notes, or configuration settings.
- **.csv (Comma Separated Values):** The industry standard for tabular data (spreadsheets). Used by almost all weather stations and yield monitors.
- **.json (JavaScript Object Notation):** Used for web-based data and IoT sensor configuration.
- **.xml / .kml:** Used for geospatial data (GIS) and mapping farm boundaries.

2 The Mechanics of File I/O in Python

To work with a file, Python follows a specific logical sequence similar to physical file management in an office: **Open** $\rightarrow$ **Work** $\rightarrow$ **Close**.

2.1 Opening Files: The `open()` Function

The `open()` function creates a ”file object” and serves as the link between your code and the storage disk.

Syntax: `file_var = open("filename.ext", "mode")`

2.1.1 File Modes

You must specify what you intend to do with the file:

- `'r'` (Read): Opens a file for reading. Error if file doesn’t exist.
- `'w'` (Write): Opens for writing. **Warning:** This overwrites the file if it already exists.
- `'a'` (Append): Opens for writing but adds data to the end of the existing content.
- `'x'` (Create): Creates the file, returns an error if it already exists.

2.2 The "With" Statement: Professional Best Practice

In the past, engineers had to remember to manually call `file.close()`. If you forgot, the file could become corrupted or the computer's memory could leak. Python 3 introduced the `with` statement (Context Manager), which **automatically closes the file** for you, even if an error occurs.

```
# The older, riskier way
f = open("log.txt", "w")
f.write("System Start")
f.close()

# THE PROFESSIONAL WAY (The 'With' Statement)
with open("daily_log.txt", "w") as f:
    f.write("2025-04-12: Irrigation started at 06:00 AM\n")
    f.write("2025-04-12: Pump pressure stable at 45 PSI\n")
# File is automatically closed here
```

3 Reading and Writing Text Files

3.1 Writing Data

When writing, Python treats the file as a long string. You must manually include newline characters (`\n`) if you want data on different lines.

3.2 Reading Data

There are three primary ways to read a file:

1. `f.read()`: Reads the entire file as one giant string.
2. `f.readline()`: Reads one line at a time.
3. `f.readlines()`: Reads the entire file and puts each line into a **List**.

```
# Reading a sensor log
with open("sensor_data.txt", "r") as f:
    all_lines = f.readlines()

for line in all_lines:
    # .strip() removes the \n from the end of each line
    print(f"Processing: {line.strip()}")
```

4 Working with CSV Files: The Agricultural Standard

Most agricultural sensors (from companies like Campbell Scientific, Trimble, or John Deere) export data in **CSV (Comma Separated Values)** format. CSV files are plain text files where each line is a row, and each piece of data is separated by a comma.

4.1 The `csv` Module

While you could parse CSVs using string methods, Python's built-in `csv` module is much more robust for handling commas inside quotes and different delimiters.

```python
import csv

# Writing to a CSV (e.g., Exporting Yield Data)
yield_data = [
    ["Plot_ID", "Crop", "Yield_tons"],
    [101, "Maize", 4.5],
    [102, "Maize", 3.8],
    [103, "Rice", 5.2]
]

with open("harvest_report.csv", "w", newline='') as f:
    writer = csv.writer(f)
    writer.writerows(yield_data)
```

4.2 Reading a CSV into Data Structures

As an engineer, you will often read a CSV and convert it into a List of Dictionaries or a List of Lists for analysis.

```python
import csv

with open("weather_station_logs.csv", "r") as f:
    # DictReader automatically uses the first row as Keys
    reader = csv.DictReader(f)

    for row in reader:
        # Access data by column name
        temp = float(row["Temperature"])
        if temp < 0:
            print(f"FROST ALERT on Date: {row['Date']}")
```

5 Google Colab: Getting Data into the Cloud

Since Google Colab runs on Google's servers, it cannot directly "see" the files on your personal computer's hard drive. You must upload them or connect to Google Drive.

5.1 Method 1: Manual Upload (Temporary)

On the left sidebar of Colab, click the **Folder Icon**. Click the **Upload Icon**. This uploads files to the temporary "Sample Data" folder. **Note:** These files are deleted when you close the session.

5.2 Method 2: Mounting Google Drive (Permanent)

This is the preferred method for engineering projects.

```python
from google.colab import drive
drive.mount('/content/drive')

# Now you can access files in your Drive like this:
# with open('/content/drive/MyDrive/AGE_219/my_data.csv', 'r') as f:
```

5.3 Method 3: Direct Upload via Code

```python
from google.colab import files
uploaded = files.upload() # This opens a file picker on your computer
```

6 Case Study 1: Parsing Weather Station Logs to Identify Frost Events

Engineering Problem: A tea plantation in Iringa has a weather station that exports a CSV every week. The engineer needs a script that automatically reads this CSV and counts how many "Frost Hours" occurred (temperature $\leq 0°C$).

```python
import csv

# Simulated CSV Data Content (Usually read from file)
# Date,Time,Temp_C,Humidity
# 2025-05-01,00:00,1.2,85
# 2025-05-01,01:00,-0.5,88
# 2025-05-01,02:00,-1.2,90

def analyze_frost(file_path):
    frost_hours = 0
    total_readings = 0

    try:
        with open(file_path, 'r') as f:
            reader = csv.DictReader(f)
            for row in reader:
                temp = float(row["Temp_C"])
                if temp <= 0:
                    frost_hours += 1
                total_readings += 1

        print(f"Analysis Complete for {file_path}")
        print(f"Total Readings: {total_readings}")
        print(f"Frost Duration: {frost_hours} hours")

    except FileNotFoundError:
        print("Error: The weather log file was not found.")
    except KeyError:
        print("Error: The CSV columns are named incorrectly. Expected 'Temp_C'.")

# analyze_frost("iringa_weather_week1.csv")
```

7 Case Study 2: Machinery Telematics Report Generator

Engineering Problem: A research tractor generates fuel consumption data. We need to create a script that reads raw readings from a list and generates a formatted text report for the farm manager.

```python
def generate_machinery_report(tractor_id, consumption_list):
    filename = f"report_{tractor_id}.txt"
    avg_fuel = sum(consumption_list) / len(consumption_list)
    max_fuel = max(consumption_list)

    with open(filename, "w") as report:
        report.write("====================================\n")
        report.write(f"FLEET PERFORMANCE REPORT: {tractor_id}\n")
        report.write("====================================\n")
        report.write(f"Number of operational hours: {len(consumption_list)}\n")
        report.write(f"Average Fuel Consumption: {avg_fuel:.2f} L/hr\n")
        report.write(f"Peak Fuel Load: {max_fuel:.2f} L/hr\n")
        report.write("------------------------------------\n")
        report.write("Generated by: AGE 219 Automation Engine\n")

    print(f"Report successfully saved as {filename}")

```

```
# Usage
tractor_data = [12.5, 14.2, 18.5, 11.0, 10.5, 22.1]
generate_machinery_report("TRACTOR_SUA_001", tractor_data)
```

8 Robust Programming: Error Handling in File I/O

File operations are "risky." The file might be missing, it might be open in another program (locking it), or the hard drive might be full. Engineers use **Try-Except** blocks to prevent the entire system from crashing when a file error occurs.

```
try:
    with open("critical_config.csv", "r") as f:
        data = f.read()
except FileNotFoundError:
    print("WARNING: Config file missing. Using factory default settings.")
    # Set default variables here
except PermissionError:
    print("ERROR: File is locked by another program.")
finally:
    print("I/O Attempt finished.")
```

9 Best Practices for Engineering Data Import

1. **Always use `with`:** Never use raw `open()` and `close()` calls.
2. **Validate Headers:** When reading CSVs, check if the column names match your expectations before processing.
3. **Use Relative Paths:** Avoid using paths like `C:\Users\John...` because they won't work on other computers. Use relative paths like `data/logs.csv`.
4. **Closing Handlers:** When writing large amounts of data, Python buffers the write. The `with` statement ensures the buffer is "flushed" to the disk correctly.
5. **Encoding:** Always assume `encoding='utf-8'` when working with text files containing special symbols (like °C).

10 Summary

In this lecture, we have bridge the gap between volatile code and the physical world of persistent data. You have learned:

- The difference between **RAM (Volatile)** and **Storage (Persistent)**.
- How to use the `with` statement to safely open, read, and write text files.
- How to parse and generate **CSV files**, the global standard for AgriTech data.
- How to manage files within the **Google Colab** environment.
- How to handle common **File I/O errors** to create robust engineering applications.

In Lecture 8, we will take these data import skills and apply them to the world's most powerful data analysis library: **Pandas**, where we will learn to handle millions of rows of agricultural data with ease.

11 Glossary of Terms

- **Append Mode (`'a'`):** A file mode that allows you to add new data to the end of an existing file without deleting its current contents.
- **CSV (Comma Separated Values):** A text file that uses a comma to separate values, used primarily for tabular data.
- **Encoding:** The system used to represent characters (letters/symbols) as numbers in a file. `UTF-8` is the modern standard.
- **File Object:** A Python object that acts as the handle or bridge to a physical file on the disk.
- **I/O (Input/Output):** The communication between an information processing system (like a computer) and the outside world.
- **Newline (`\n`):** A special character used in strings to signal the start of a new line in a text file.
- **Persistence:** The characteristic of data that outlives the process that created it.
- **Volatile Memory:** Computer memory that requires power to maintain the stored information (RAM).

12 Extended Self-Assessment Exercises

12.1 Exercise 1: Simple Logger

Write a program that asks the user to type in three soil moisture readings. Save these readings into a file named `moisture_log.txt`, each on a new line. Then, write a second part of the program that reads this file and prints the average of the three readings.

12.2 Exercise 2: Field Coordinate Export

You have a list of boundary coordinates for a circular irrigation plot: `coords = [(0,0), (10,10), (20,0), (10,-10)]` **Task:** Write a script using the `csv` module to save these coordinates into a file named `plot_boundary.csv` with headers "X_Coord" and "Y_Coord".

12.3 Exercise 3: The Corrupted Log Recovery

A weather station log (`weather.csv`) has some missing data points (empty strings) in the "Rainfall" column. **Task:** Write a script that reads the file, identifies which rows have missing data, and prints the Date of those rows. Use a `try-except` block to handle the case where the file might be missing.

12.4 Exercise 4: Colab Path Logic

Explain in three sentences why you cannot use a file path like `D:/MyDocuments/data.csv` inside a Google Colab notebook, and describe the correct way to access that file.

End of Lecture 7 Compendium